2024-26

Internet of Things (IoT) Design and Development

Assignment – 1

Smart Home Energy Monitoring Control and System

By

Ganesh. T - 2024HT01538
Vivek. K - 2024HT01668
Abhilash K - 2024HT01650

Objective:

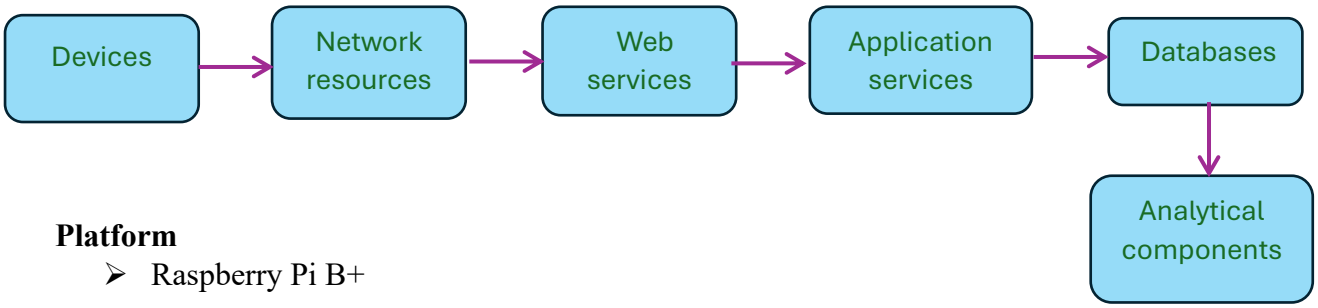
To design an IoT-based smart two-room system that monitors energy usage and automatically controls appliances such as light, fan, and air conditioner based on human presence, while also allowing manual control through a touchscreen interface with user override capability, and ensuring continuous operation of essential devices like the refrigerator.

Introduction

The rapid growth of Internet of Things (IoT) technologies has transformed traditional living spaces into intelligent, connected environments capable of sensing, communicating, and acting in real time. Energy consumption in homes is increasing due to the widespread use of electrical appliances, making efficient energy management an essential requirement for both sustainability and cost reduction. A Smart Home Energy Monitoring and Control System leverages IoT concepts to provide real-time visibility, automation, and intelligent decision-making for optimizing household power usage.

This project focuses on designing a real-time embedded IoT application that monitors, analyses, and reports energy usage within a smart home environment. Using embedded systems principles, the project integrates an embedded operating system, suitable programming languages, onboard sensors, and a lightweight database to collect and store consumption data. IoT communication protocols enable seamless connectivity between devices, sensors, and cloud platforms allowing remote monitoring, control, and analytics through web or mobile interfaces.

By combining embedded systems with IoT connectivity, the system enhances user awareness of power utilization patterns, supports energy-saving decisions, and enables automation such as scheduling, remote appliance control, and anomaly detection. Ultimately, this project demonstrates how embedded IoT solutions can contribute to smarter homes, lower energy bills, and sustainable energy usage through efficient real-time monitoring and intelligent control.



Platform

- Raspberry Pi B+

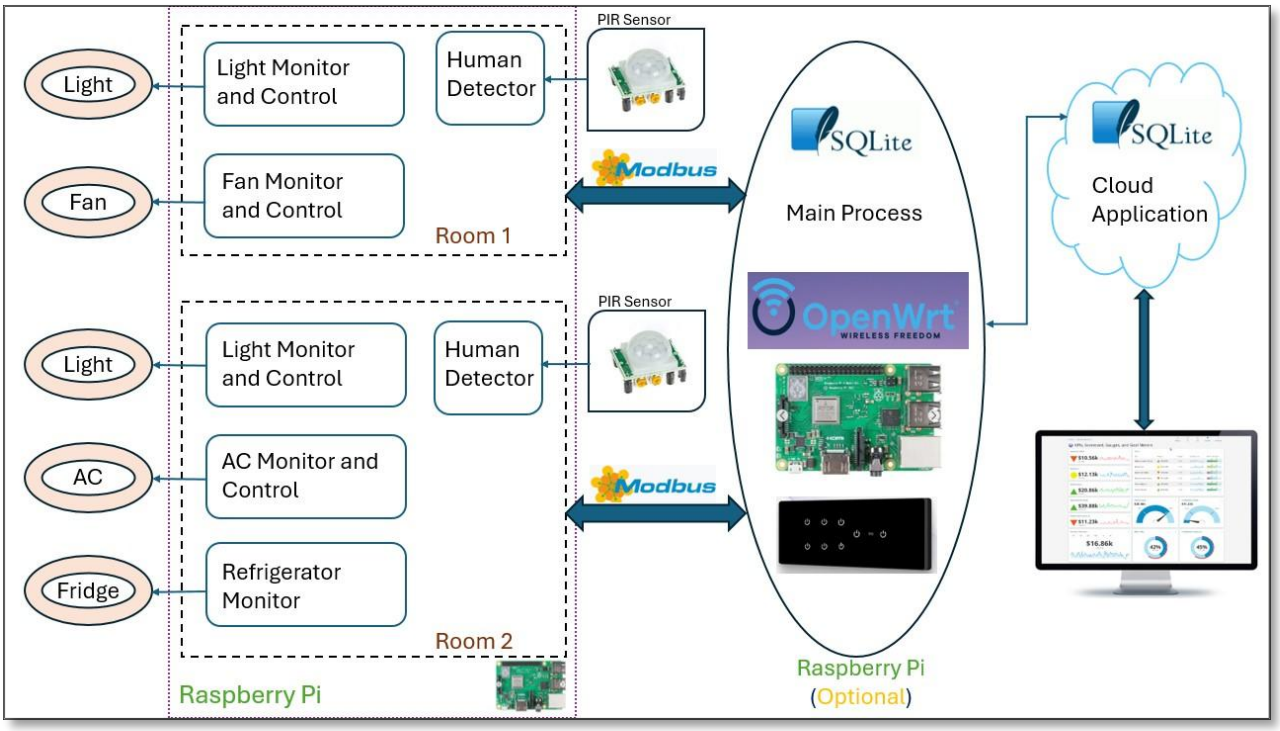
Protocols

- Modbus
- MQTT

Tools and Software

- Python (Web dashboard)
- SQLite
- MQTT (Mosquitto)
- Modbus library
- Simple UI - LVGL

Block Diagram



These are the software-based Power Sensor Simulators that emulate energy-consuming of appliances, and it controls,

- **Module 1:** Light Monitor and Control
- **Module 2:** Fan Monitor and Control
- **Module 3:** Air Conditioner Monitor and Control
- **Module 4:** Refrigerator Monitor

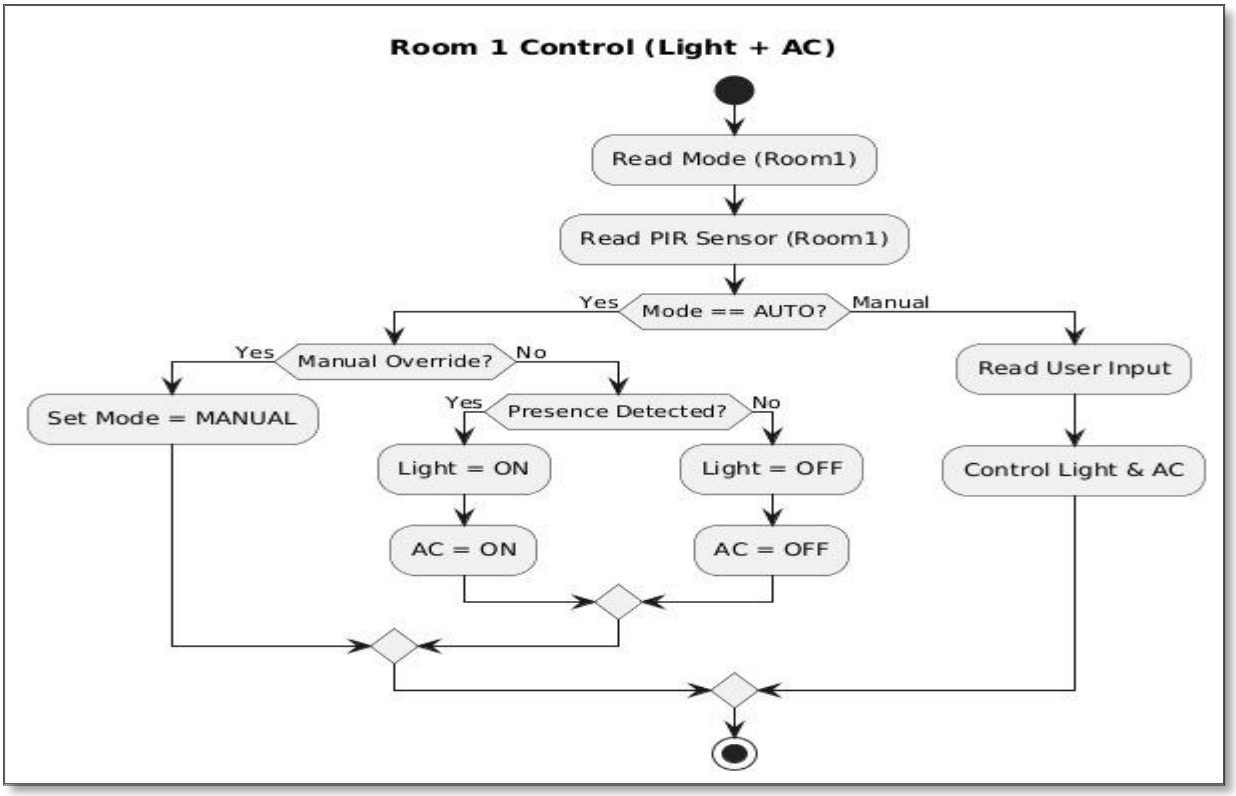
IOT -A reference model

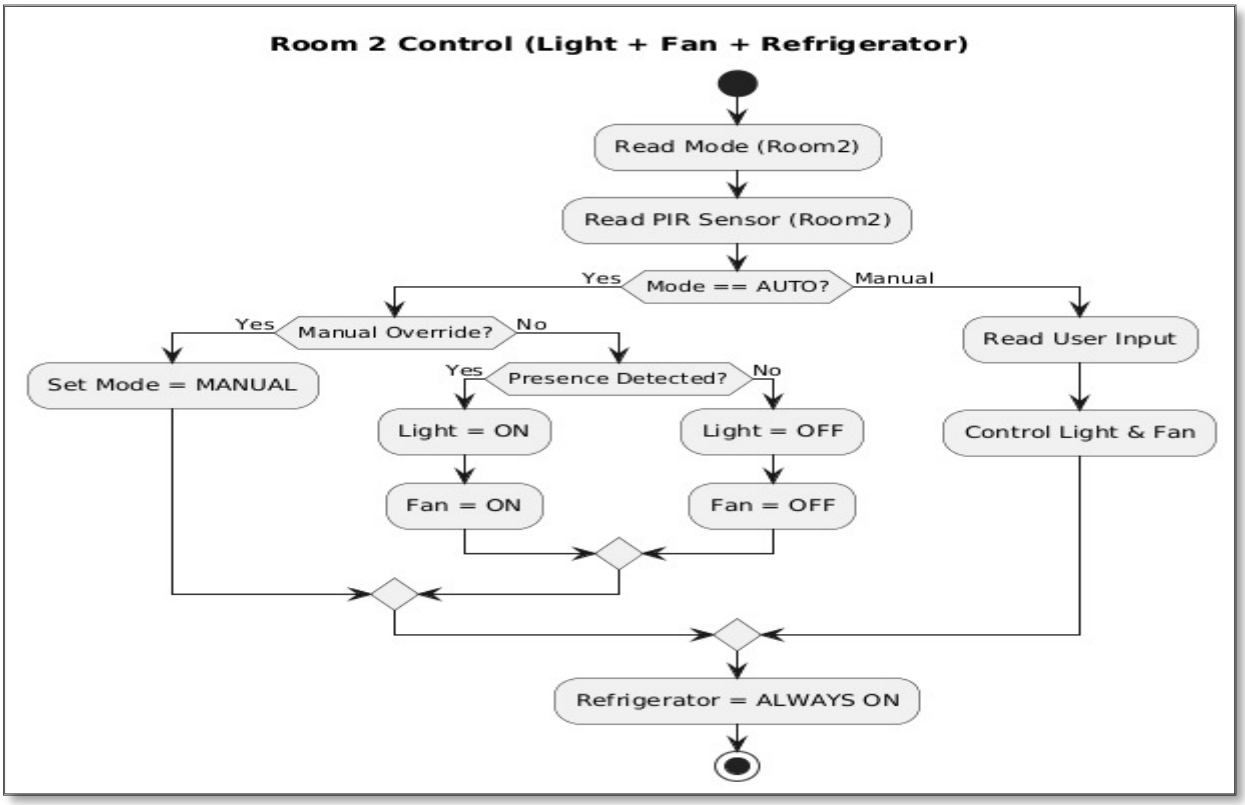
We are going to design methodology that is called IOT -A reference model. It takes top-down design approach. There are 10 steps to do design, they are

1. Purpose and requirement specification
2. Process model specification
3. Domin model specification
4. Information model specification
5. Service specification
6. IOT level specification
7. Functional view specification
8. Operational view specification
9. Device and components integration
10. Application development

1. Purpose and requirement specification

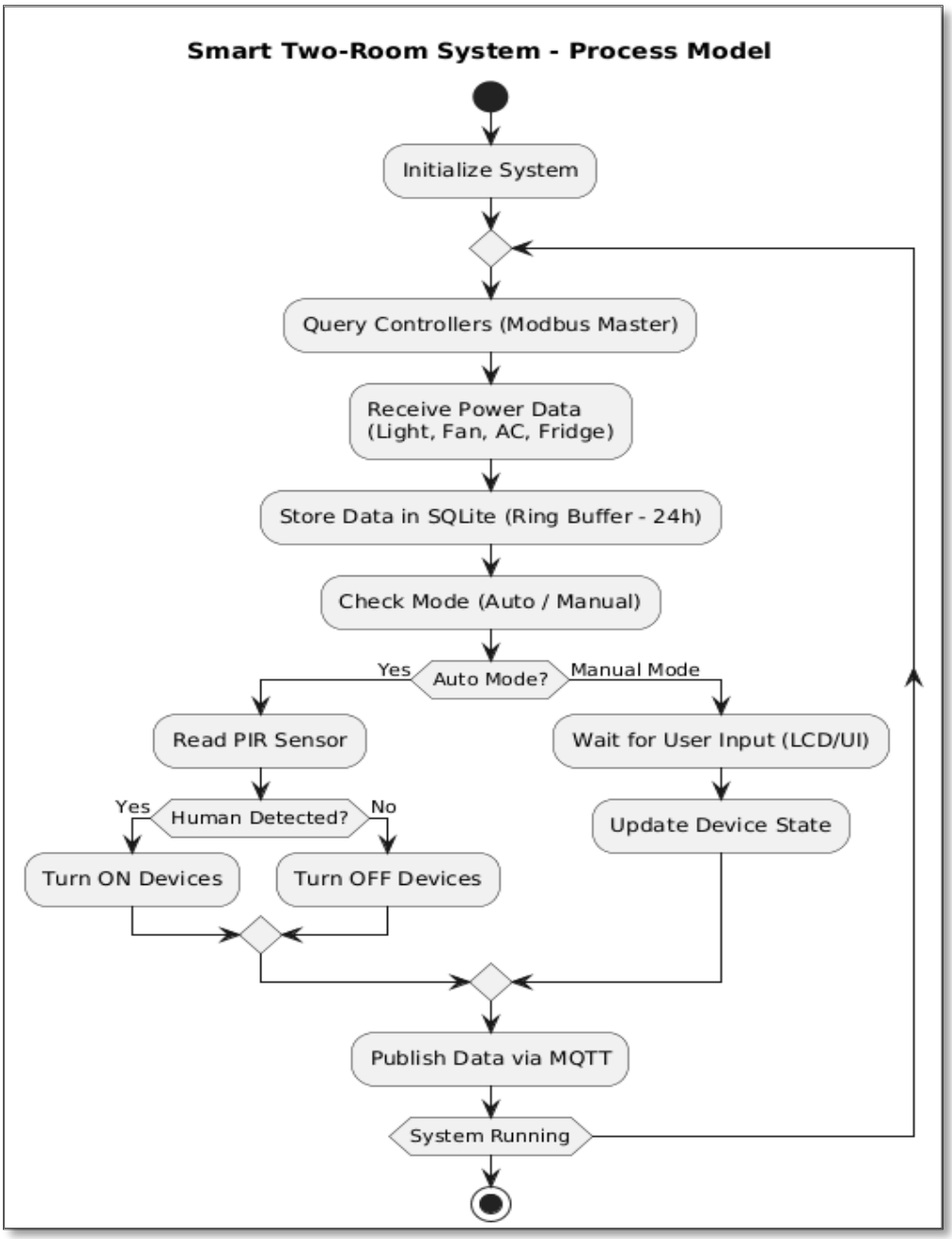
We will be defining the purpose of the system, the behaviour of the system. What are the requirements of the various components which would be part of the IoT system. At this point, we will be able to capture complete requirement of the IoT system.





2. Process model specification

We will define all use cases of the IoT systems in this stage. The system periodically collects simulated power data using Modbus, detects human presence using PIR sensors, applies auto or manual control logic, stores data using a ring buffer, and publishes it to the cloud using MQTT.



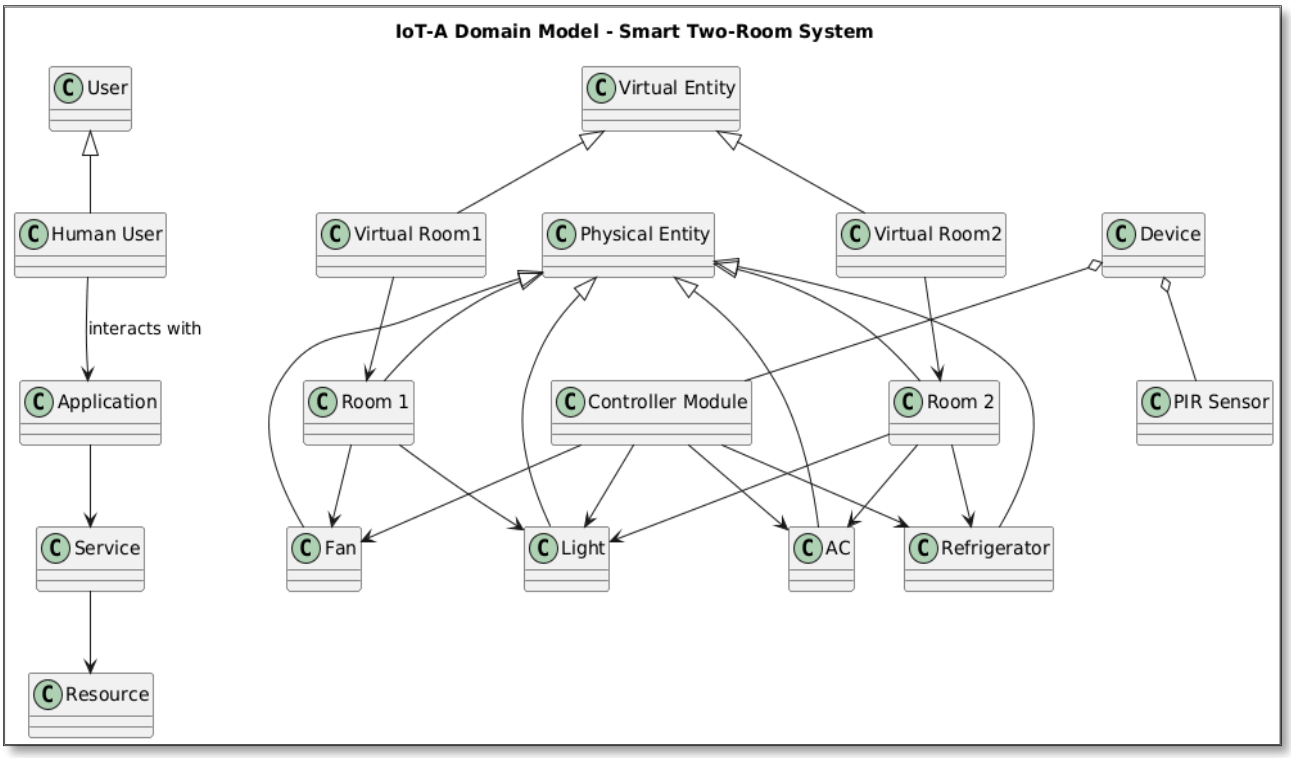
Control Logic Execution

- In Auto Mode:
  - If human detected - Turn ON devices
  - If no human - Turn OFF devices
- In Manual Mode:
  - Wait for user input from LCD or cloud
- If user overrides in Auto Mode:
  - System switches to **Manual Mode**

3. Domin model specification

Here we define all entities, the resources, The devices. Entity could be physical and virtual entities. We would design that what are the physical devices. We will define that what are the various services. All the components are within the IoT domine are going to specified in the IoT domain model specification.

The domain model represents the structure of the system, including users, devices, physical and virtual entities, and how they interact.



User & Application

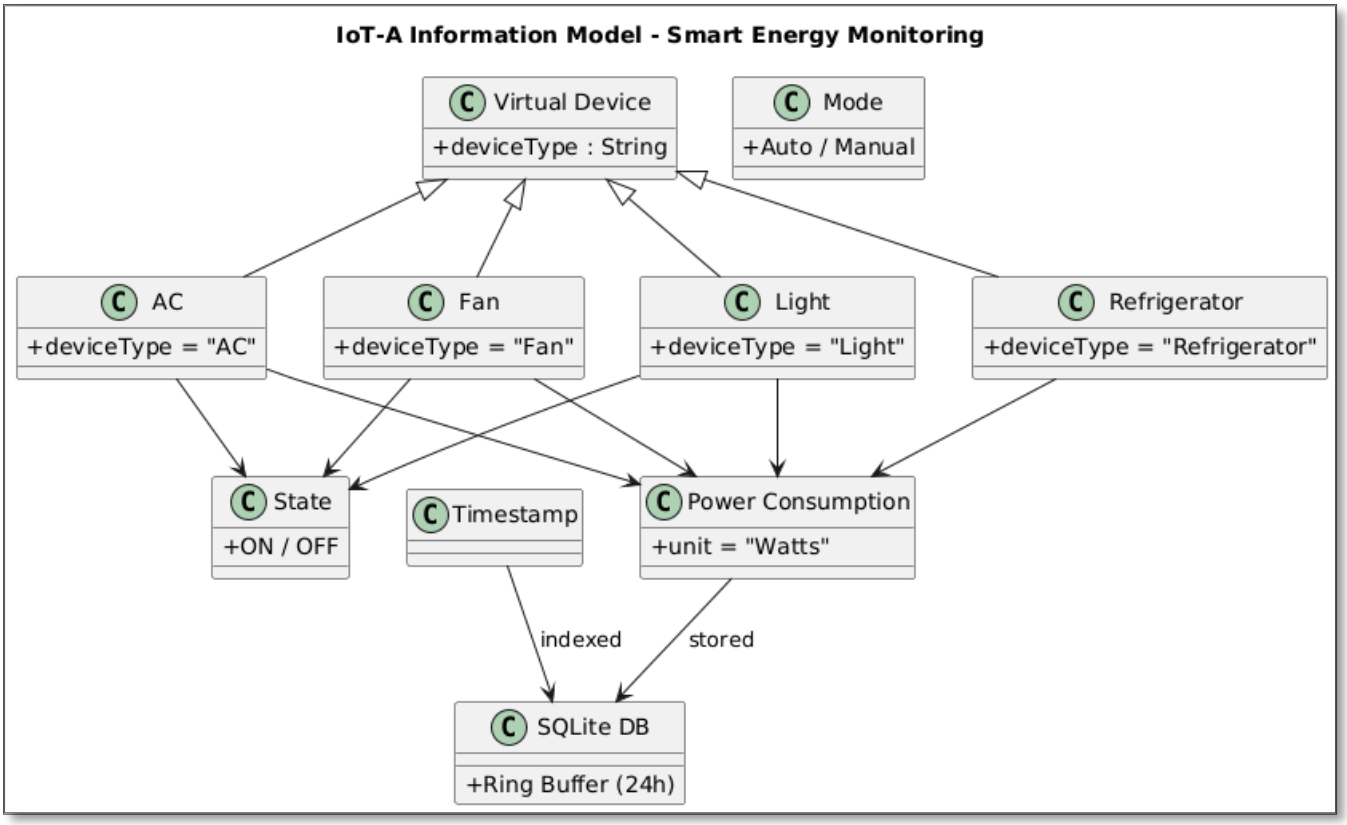
- The system includes a **Human User**
- User interacts with:
  - **Touch LCD (local control)**
  - **Web Dashboard (remote control)**
- Application provides:
  - Monitoring
  - Device control

Services & Resources

- Services provide system functionality:
  - Control service
  - Data service
- Resources represent:
  - Device state
  - Power data
  - Mode (Auto/Manual)

4. Information model specification

We are going to define the structure all the information within the system. The structure would the relation between various modules, the attributes of the various modules,



Device Information

- Each device (Light, Fan, AC, Refrigerator) is represented as a data entity
- Attributes include:
  - Device Type
  - Device ID
- Used for identification and monitoring

Mode Information

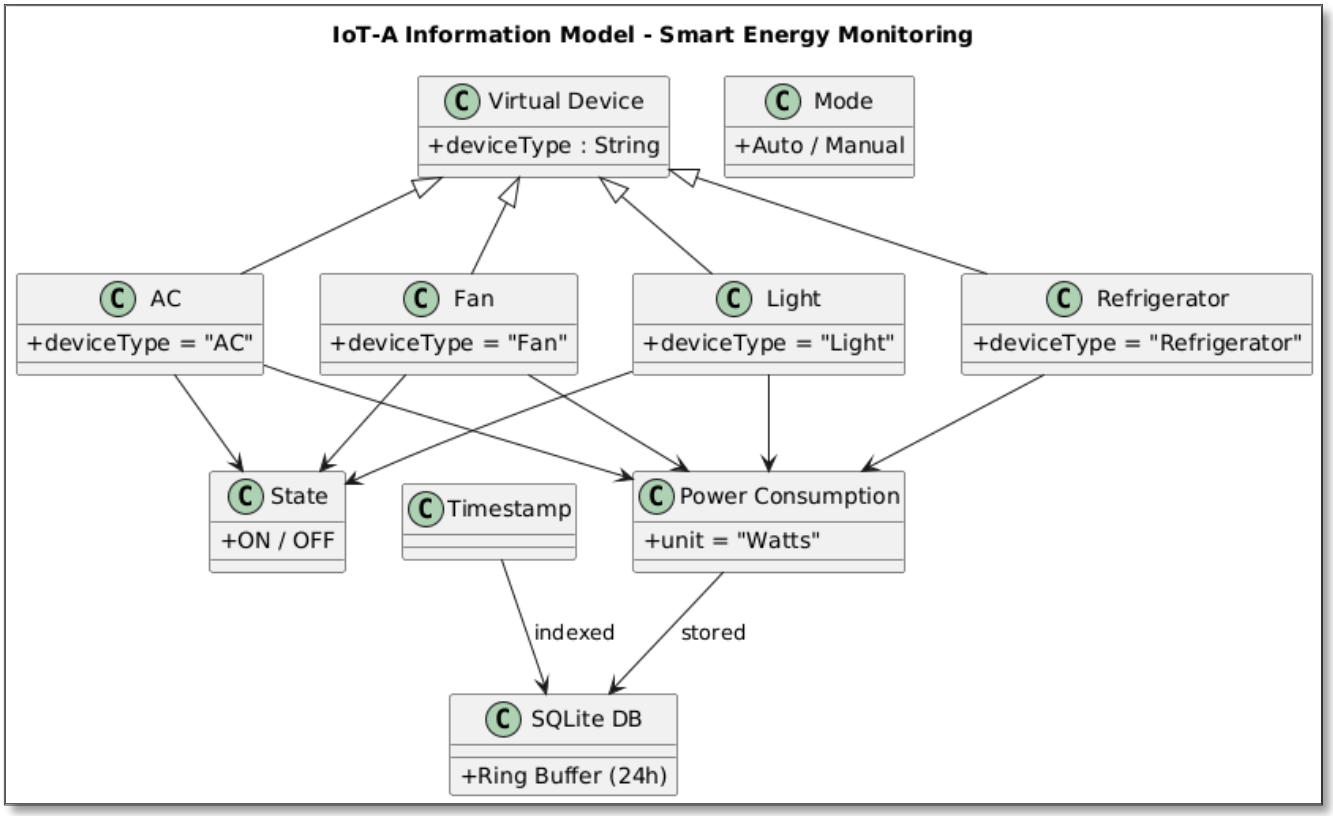
- Defines system operation mode:
  - **Auto Mode** - Controlled by PIR sensor
  - **Manual Mode** - Controlled by user
- Supports **manual override**, where auto mode switches to manual when user intervenes

Presence Information

- Obtained from PIR sensors
- Represents:
  - Human Detected
  - No Human
- Used for decision-making in auto mode

5. Service model specification

It takes the process model specification and information model specification, them maps those into different services which are going to available. This will give the specification of the services.



MQTT Communication Service

- Publishes data to cloud application
- Data includes:
  - Power consumption
  - Device state
- Operates periodically

User Interface Service

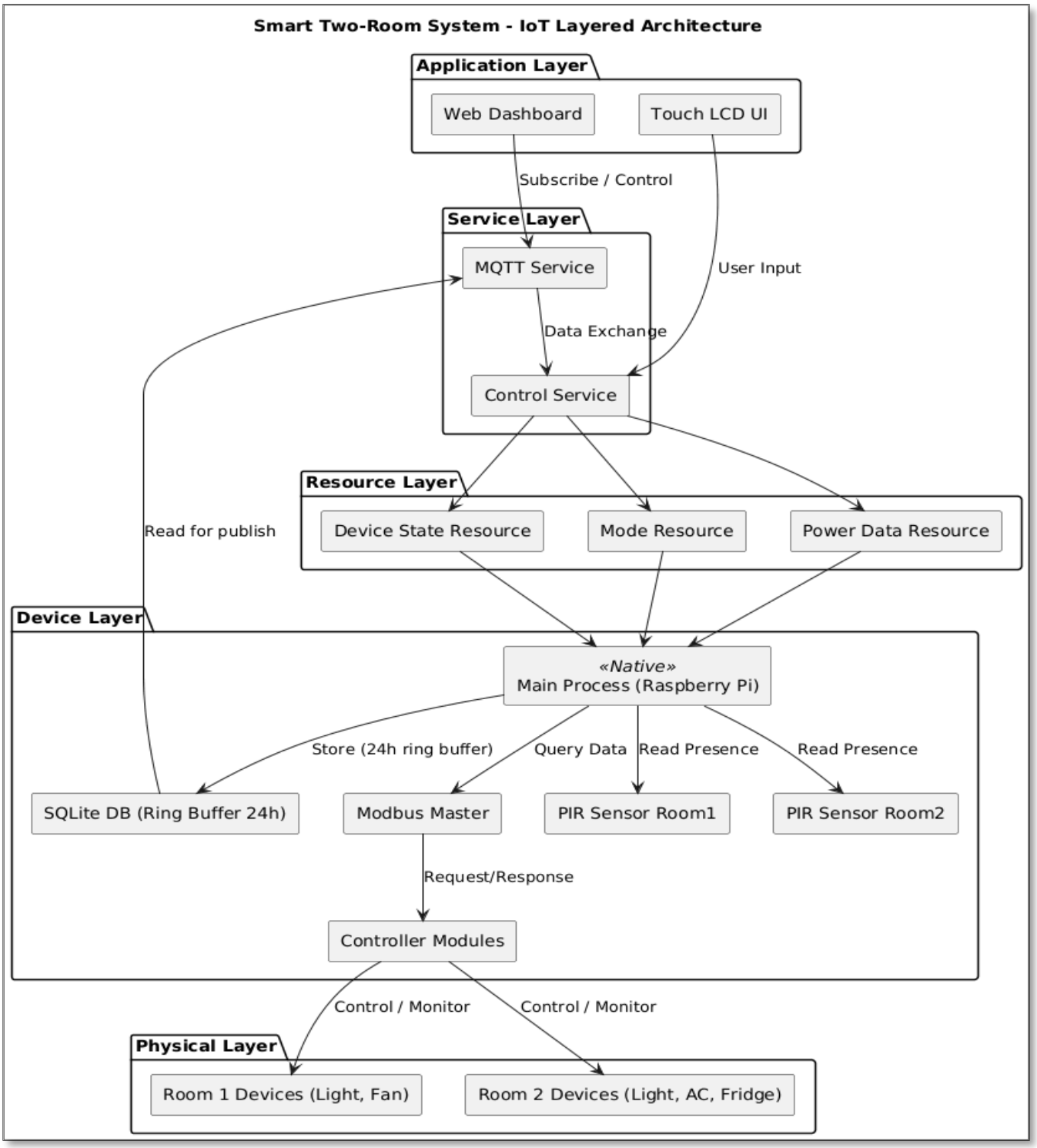
- Provides interaction through:
  - Touch LCD (local control)
  - Web dashboard (remote control)
- Allows:
  - Device ON/OFF control
  - Mode selection

Cloud Service

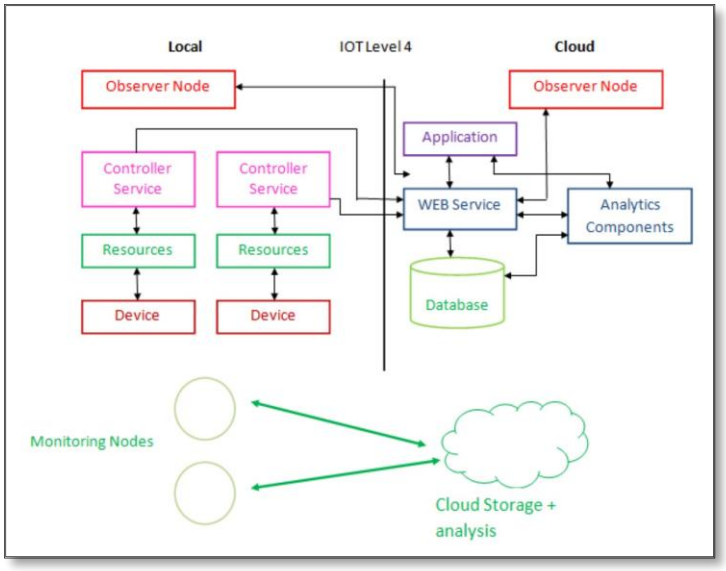
- Receives data via MQTT
- Stores data in cloud database
- Provides:
  - Visualization (dashboard)
  - Remote control functionality

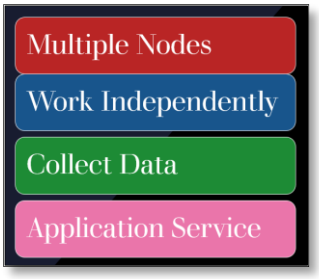
6. IoT level specifications (Level 4)

Here will say that where the IoT application fall under level one to six. It is depending on up one the component which are going to be local or remote. We can decide that what is the level of IoT.



The IoT level specification defines the **deployment architecture and interaction pattern** of the system. This project follows the **IoT Level 4 architecture**, which includes both **local processing and cloud integration** for data storage, analytics, and user interaction.





Benefits

- **Resilience:** Local control continues even if the WAN link fails.
- **Low latency:** Immediate response for occupancy-based actions.
- **Scalability:** Cloud handles many homes/sites and long-term analytics.
- **Security boundary:** Internet-facing endpoints limited to cloud; local network remains shielded.

Trade-offs

- **More moving parts:** Local + cloud components to deploy and maintain.
- **Sync complexity:** Must handle **offline buffering**, retries, and **conflict-free updates**.
- **Security management:** Certificates/keys both locally and in cloud.

Key Characteristics of Level 4

- Edge device performs **data collection and control logic**
- Cloud platform provides **storage, visualization, and remote access**
- System supports **both local and remote control**
- Communication uses **internet protocols (MQTT)**

System Mapping to IoT Level 4

1. Edge Layer (Device Level)

- Raspberry Pi (Main Controller)
- PIR Sensors (Presence Detection)
- Controller Modules (Simulated Power Data)
- LEDs (Device simulation: Light, Fan, AC, Refrigerator)

Responsibilities:

- Data acquisition (Modbus)
- Control logic (Auto/Manual + Override)
- Local decision making

2. Gateway & Processing Layer

- Raspberry Pi acts as:
  - **Processing unit**
  - **Gateway device**

Responsibilities:

- Data processing
- Data storage (SQLite, 24-hour ring buffer)
- MQTT communication

3. Cloud Layer

- Cloud Application + Database

Responsibilities:

- Data storage
- Dashboard visualization
- Remote control

4. Application Layer

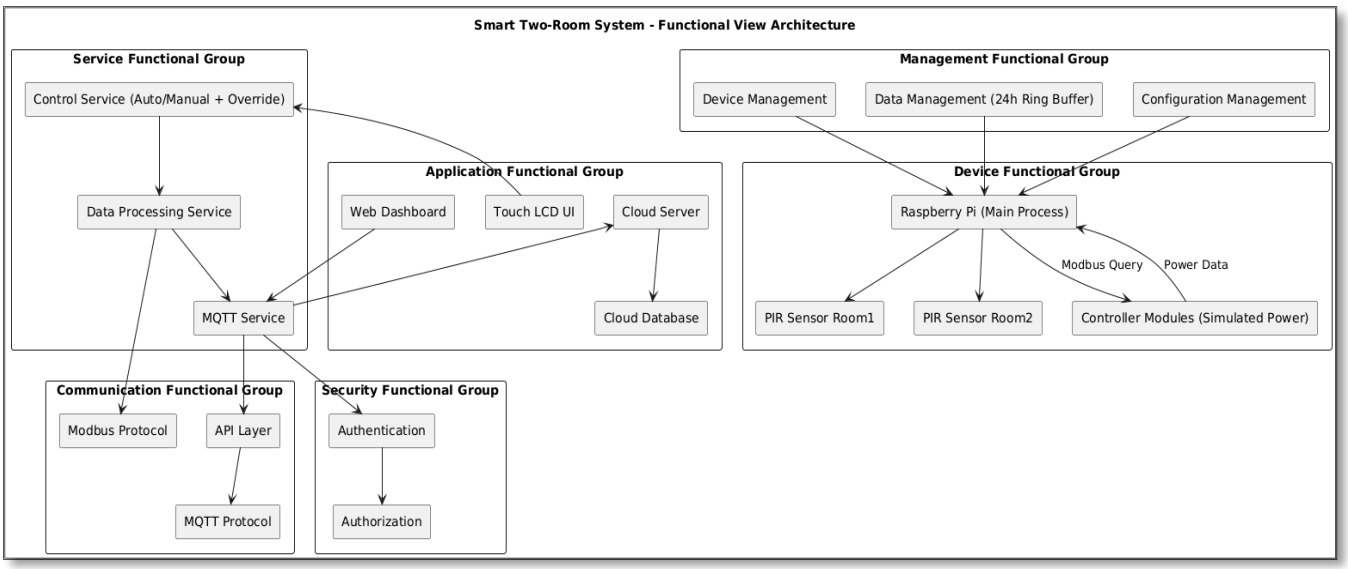
- Web Dashboard
- Touch LCD Interface

Responsibilities:

- User interaction
- Monitoring and control

7. Functional view specification

This is going to map IoT level will have multiple functions. So, the functions are in the IoT level going to be classified into functional groups. It defines what are the various functions under each function. Also provides the characteristics of these functions.



The functional view organizes the system into groups such as application, service, device, communication, management, and security, and describes how these groups interact to perform system operations.

The Functional View Specification describes the **functional components (groups)** of the IoT system and how they interact to perform the overall system operation.

It focuses on:

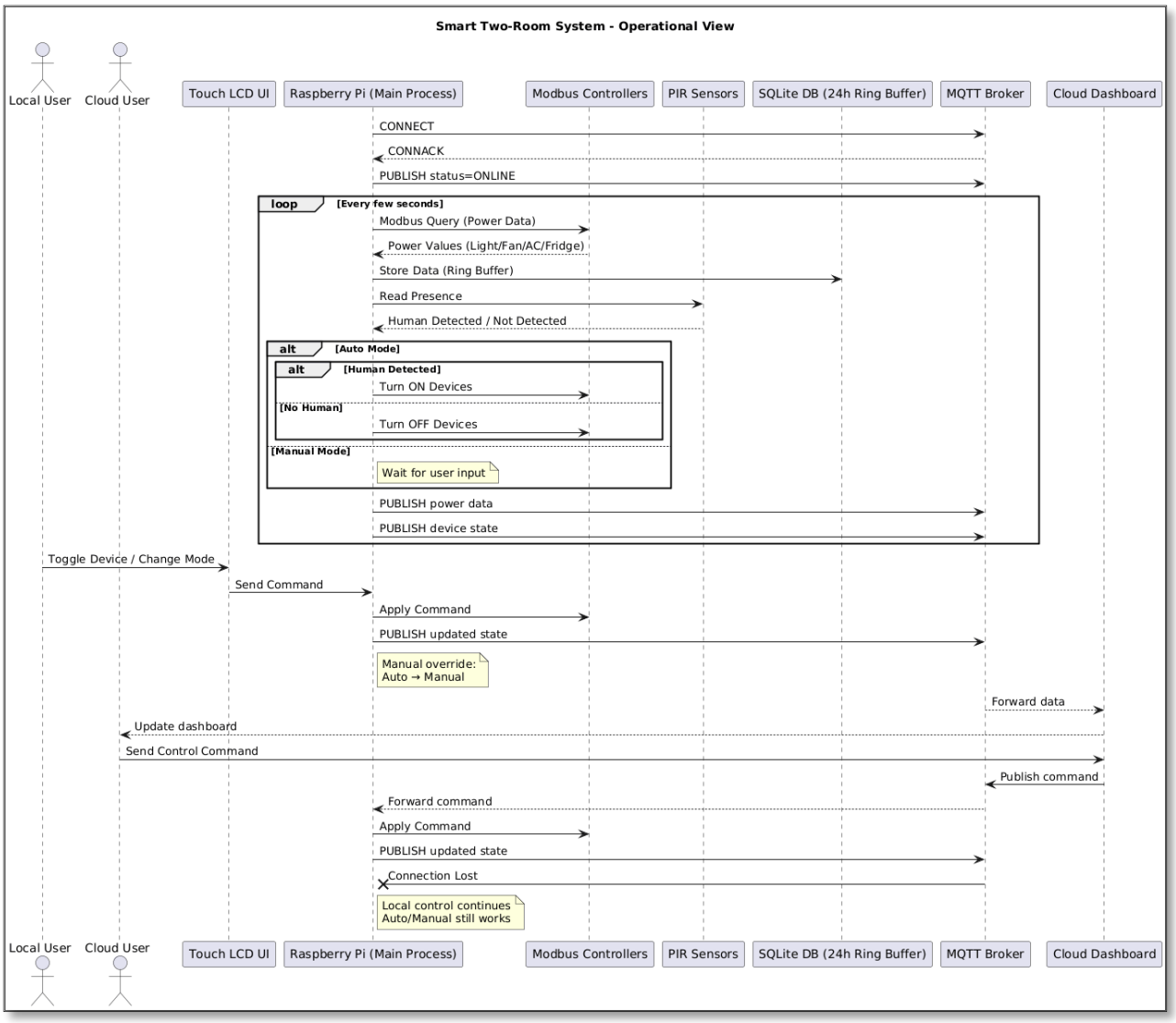
- What functions are performed
- How components are logically grouped
- Interaction between functional groups

Interaction Between Functional Groups

- Application Group - interacts with Service Group
- Service Group - communicates with Device Group
- Device Group - sends data via Communication Group
- Communication Group - connects to Cloud
- Management Group - monitors system behaviour
- Security Group - protects all interactions

8. Operational view specifications

Here we are going to provide all the options possible. Options in terms of communication, service hosting, storage options, the various device which are going to specify. So essentially what we will be doing is we will be selecting various components of the IoT system.



System Initialization

- Raspberry Pi starts the main process
- Initializes:
  - Modbus Master
  - SQLite Database
  - MQTT Client
- Establishes connection with cloud MQTT broker
- Publishes system status as **ONLINE**

Data Acquisition (Periodic Operation)

- Main process sends Modbus requests to:
  - Light Controller
  - Fan Controller
  - AC Controller
  - Refrigerator Monitor
- Controllers respond with:
  - Simulated power consumption values

Local Data Processing

- Received data is:
  - Processed and categorized
  - Checked for validity
- Control logic may be applied if needed

Data Storage Operation

- Data is stored in **SQLite database**
- Uses **24-hour ring buffer mechanism**:
  - Stores only latest 24 hours data
  - Old data is overwritten automatically

Cloud Communication

- MQTT client publishes:
  - Power consumption data
  - Device status
- Data sent periodically to cloud topics

Dashboard Monitoring

- Cloud application:
  - Receives MQTT data
  - Stores in cloud database
- Web dashboard displays:
  - Real-time power usage
  - Historical data

User Control Operation

- User interacts via:
  - Web dashboard
  - Touch LCD
- Control commands:
  - Device ON/OFF
  - Mode selection
- Commands are sent to:
  - Cloud - MQTT - Raspberry Pi
  - OR directly via local interface

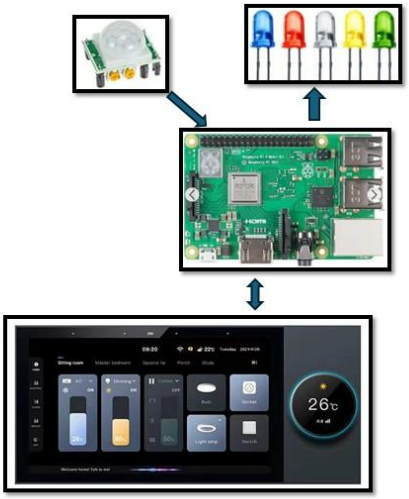
Control Execution

- Raspberry Pi receives command
- Updates device state
- Sends control signal to:
  - Corresponding controller module
- Updates status back to cloud

Internet Failure Handling

- If MQTT connection fails:
  - Local system continues operation
  - Data collection and control still active
- Cloud sync resumes once connection is restored

9. Device and component integration



We are going to build different components and integrate components together. The Device and Component Integration section describes how **hardware devices, software modules, and communication interfaces** are integrated to form a complete IoT system.

- Interconnection of components
- Data flow between devices
- Integration of protocols and services

Key Components in the System

1. Edge Device (Main Controller)

- **Raspberry Pi**

  Role:

  - Acts as:
    - Modbus Master
    - Data processing unit
    - MQTT client
  - Central controller of the system

2. Controller Modules

- Light Controller & Monitor
- Fan Controller & Monitor
- AC Controller & Monitor
- Refrigerator Monitor

  Role:
  - Generate **simulated power consumption data**
  - Respond to Modbus queries from Raspberry Pi

3. Communication Interfaces

Modbus Communication

- Used between:
  - Raspberry Pi (Master)
  - Controller Modules (Slaves)

  Function:
  - Transfer power consumption data
  - Request-response based communication

MQTT Communication

- Used between:
  - Raspberry Pi
  - Cloud Application

  Function:
  - Publish:
    - Power data
    - Device status
  - Receive:
    - Control commands

4. Local Storage Component

- **SQLite Database**

  Role:
  - Stores system data locally
  - Implements:
    - **24-hour ring buffer**
  - Ensures efficient memory usage

5. Cloud Components

- MQTT Broker
- Cloud Database
- Web Dashboard

  Role:
  - Data storage
  - Visualization
  - Remote access and control

6. User Interface Components

- Touch LCD (Local Control)
- Web Dashboard (Remote Control)

  Role:
  - User interaction
  - Monitoring and control

Integration Architecture

- Raspberry Pi acts as the **central integration hub**
- All components communicate through defined protocols
- Modular design allows easy addition of new devices

10. Application development

This section explains how the **end user interacts with the IoT application**, including both **local and remote access methods**. The system is designed to provide **easy, real-time monitoring and control** of power consumption devices.

Application Interfaces

1. Local Access (Touch LCD Interface)

Description:

- A **touch-based LCD interface** connected to the Raspberry Pi
- Provides direct control without internet dependency

User Capabilities:

- View:
  - Device status (ON/OFF)
  - Power consumption values
- Control:
  - Light, Fan, AC devices
- Select:
  - Operating modes (Auto / Manual)

Advantages:

- Works even during **internet failure**
- Fast response (no cloud delay)
- Simple and user-friendly

2. Remote Access (Web Dashboard)

Description:

- A **web-based application** hosted on the cloud
- Accessible via:
  - Mobile
  - Laptop

User Capabilities:

- Monitor:
  - Real-time power consumption
  - Historical data (last 24 hours)
- Control:
  - Device ON/OFF
  - Mode selection
- Visualize:
  - Graphs and usage trends

Access Workflow

Remote Access Flow:

1. User opens web dashboard
2. Dashboard connects to cloud server
3. Cloud retrieves data from database
4. Displays real-time device data
5. User sends control command
6. Command sent via MQTT
7. Raspberry Pi receives and executes command

Local Access Flow:

1. User interacts with Touch LCD
2. Input sent directly to Raspberry Pi
3. Raspberry Pi processes command
4. Device state updated instantly

Communication Mechanism

- **HTTP/REST** - Web dashboard communication
- **MQTT** - Cloud - Raspberry Pi
- **Modbus** - Raspberry Pi - Controllers

Conclusion

The proposed IoT-based power monitoring system successfully integrates local processing and cloud communication for efficient device control and monitoring. It enables real-time data acquisition, visualization, and user interaction through both local and remote interfaces. The system is scalable, reliable, and suitable for smart home energy management applications.